Unit Name:	Cloud Computing Architecture		
Unit Code:	11368		
Semester:	2	Year:	2026
Tutorial Day & Time:	Thursday - 1:30		
Assessment Name:	11368 Cloud Computing Architecture Group Cloud Solution Project		

We declare that this assessment is solely our own work, except where due acknowledgements are made. We acknowledge that the assessor of this assessment may provide a copy of this assessment to another member of the University, and/or to a plagiarism checking service whilst assessing this assessment. We have read and understood the University Policies in respect of Student Academic Honesty.

Table of Contents

Scenario: Community Sports Club Portal **3**
Design constraints 3
Tables **4**
Assumption Table 6
Requirement Table **8**
Testing Table **10**
Report **12**
Requirements 12
Architecture Diagram **12**
Chosen Services 13
Testing Results 13
Monthly Estimates 13
Pillars 13
Recorded presentation, 12 minutes, with slides 14
Individual reflection, 400 words each **14**
Haider: 14
Zubair: 14
Ben: 14

Scenario: Community Sports Club Portal

Description:
A regional sporting association covers 12 clubs and about 4,000 members, with no IT staff of its own. Registrations, annual renewals and court bookings currently run on a set of spreadsheets and a shared email inbox, and the volunteer committee has decided it can no longer keep up. They want members doing all of that themselves through a portal, including uploading a profile photo. Photos come straight off phones and are large, and the committee does not want members watching a spinner while something resizes them. Almost the whole year's traffic arrives on the two or three Monday evenings when registration opens. Under the association's constitution, member records have to be kept for seven years, though nobody has suggested the photos need keeping that long.

1. Requirements:

Peak load: almost all traffic arrives on two or three Monday evenings. We sized for all the worst cases: All 4000 members active at once, one request each every 10 seconds, which is 400 requests per second (Assumption 6). R9 asks for under 1% of errors and a 95% response time under 2 seconds at that load. Because the evenings are predictable, the template also raises the minimum server count every Monday (assumption 9)

Photos: Phone photos run 5 to 10 MB (assumption 3), members must not wait for a resize so the upload can confirm (R3), and a background job produces a copy of at most 512 pixels within 60 seconds (R4). a failed job is retired three times, then parked in a dead letter queue that raises the alarm, so no upload is lost (R5). Raw photos expire after 30 days and are resized after 400, just over a year, so a photo survives until the next renewal; neither goes to Glacier (R7, A12).

Bookings: Member completes for the same courts when the registration opens, so 2 simultaneous requests for one slot must produce exactly one booking (R2). We defined a slot as court date and hour (A1) and made the database enforce it with a unique constraint (assumption 2) so a race in application code cannot create a double booking.

Records: The constitution requires 7 years of member records, which we read as 2,557 days (R6). Records live in RDS and AWS backup keeps a monthly recovery point for that period in a vault that survives deletion for the stack.

Operations and Lab limits: With no IT staff, the system must replace a failed server by itself (R11) and email plain English alerts (R16). The lab caps us at 9 instances and 32 vCPUs, so the scaling group stops at 6, leaving room for a load generator (R10). Lab RDS cannot be Multi AZ,

so we accept up to 60 minutes of database downtime and 24 hours of data loss outside registration evenings (R12, Assumption7)

Tables

Source Key:

Tag	Meaning
Scn	A sentence in the scenario
DC1	“The whole solution deploys from a single infrastructure as code template. A console-built solution does not satisfy this constraint.”
DC2	“The request-handling tier runs across two Availability Zones behind a load balancer, with a scaling policy on a metric of your choosing.”
DC3	“No compute instance or database is reachable from the public internet.”
DC4	“Persistent data is held in managed AWS services.”
DC5	“At least one processing path is asynchronous, so that slow or bursty work does not block a user.”
DC6	“Credentials are held in a secrets service and appear nowhere in your code or template.”
DC7	“The solution reports on itself through a dashboard and at least one alarm.”
Lab	A Learner Lab restriction
Sub	The submission section of the brief
Team	A team decision

Requirements Table:

No	Portal Requirements	Source	Assumption
1	A member can register, renew annually, book a court and upload a profile photo through the portal with no committee member or manual intervention involved	Scn: “Registrations, annual renewals and court bookings currently run on a set of spreadsheets and a shared email inbox, and the volunteer committee has decided it can no longer keep up. They want members doing all of that themselves through a portal, including uploading a profile photo.”	A1, A10, A11
2	When two members submit a booking for the same court and time slot simultaneously, exactly one booking is confirmed and the other receives a clear “slot unavailable” message, so no double-booking is ever stored	Derived from Scn: “Almost the whole year's traffic arrives on the two or three Monday evenings when registration opens”, so many members will compete for the same slots at once	A1, A2, A4, A5
3	The portal confirms that it has received an uploaded photo within 2 seconds of the file reaching the server/S3, with no resize required in the request	Scn: “does not want members watching a spinner.” and DC5	A3
4	A resized photo with a maximum 512 pixels on its long end is available within 60 seconds of upload via background work	DC5	A3

No	Portal Requirements	Source	Assumption
5	If photo processing fails, the job is retried 3 times and then moved to a dead-letter queue. No uploads will be lost and an alarm will fire	DC5, and Sub: testing must include “the asynchronous path including a failure on it”	A3
6	Member records are retained for at least 7 years (2557 days), nothing expires them earlier. It will live in RDS, and AWS backup takes a monthly recovery point that is retained for 2,557 days in a backup vault which is kept even if the stack is deleted	Scn: “member records have to be kept for seven years” Using AWS backup for the monthly recovery point was a team decision	A5, A11
7	Raw uploaded photos are deleted automatically after 30 days once the resized version exists. The resized photo is kept while the membership is active and deleted 400 days, and photos are never moved to the Glacier	Scn: “member records have to be kept for seven years, though nobody has suggested the photos need keeping that long.”. The durations are a team decision	A3, A12
8	The ALB distributes traffic to healthy instances in both availability zones, and during a load test, both AZ’s will show traffic	DC2	A6

No	Portal Requirements	Source	Assumption
9	Under A6 peak load (4000 concurrent sessions, around 400 requests per second), the portal keeps the error rate under 1%, and 95% of request's response times are under 2 seconds. The ASG scales out when the average CPU usage exceeds 60% or when requests per instance exceeds 300 and returns to baseline within 30 minutes after the load has ended	Scn: " Almost the whole year's traffic arrives on the two or three Monday evenings when registration opens.", DC2	A6, A9, A10
10	The solution never exceeds 9 running EC2 instances or 32 vCPU and no instance is bigger than "large". The ASG maximum is 6, leaving room for the k6 load generator if it runs on EC2, and RDS is no larger than medium	Lab: "EC2 is capped at nine running instances and 32 vCPU, with nothing above a large instance type, and RDS goes up to medium."	A6, A9
11	If one app instance gets terminated, the portal stays available and a replacement comes into service within 5 minutes	DC2, Sub: testing must cover "what happens when a component dies"	A6
12	The database is single-AZ, and if it fails, it can be restored within 60 minutes with at most 24 hours of data loss. The procedure is going	The request tier spans across two availability zones, which follows DC2, however the database runs as a single-AZ instance because the lab does not support Multi-AZ RDS. If the database's AZ fails, the app tier	A7

No	Portal Requirements	Source	Assumption
	to be documented and tested once	stays up and database is restored within 60 minutes with at most 24 hours of data loss	
13	No EC2 instance or database will have a public IP address or will be reachable from the internet. Only the ALB accepts internet traffic, the app tier only accepts traffic from the ALB's security group, and the database only accepts traffic from the app tier	DC3, Sub: testing must cover "whether anything is reachable that should not be"	None
14	No credentials appear in the template, code or repo. The app retrieves database credentials from the secrets manager at runtime	DC6	None
15	All persistent data such as member records, bookings, and photos, are held in managed in AWS (RDS/S3) and are encrypted at rest. Nothing durable lives on instance disks. Traffic between members and the ALB is HTTP-only because the lab doesn't provide a domain or certificate	DC4, Lab: no domain or certificate. Encryption at rest is a team decision	A5
16	A CloudWatch dashboard shows request counts, CPU	DC7, Scn: "no IT staff of its own"	A8

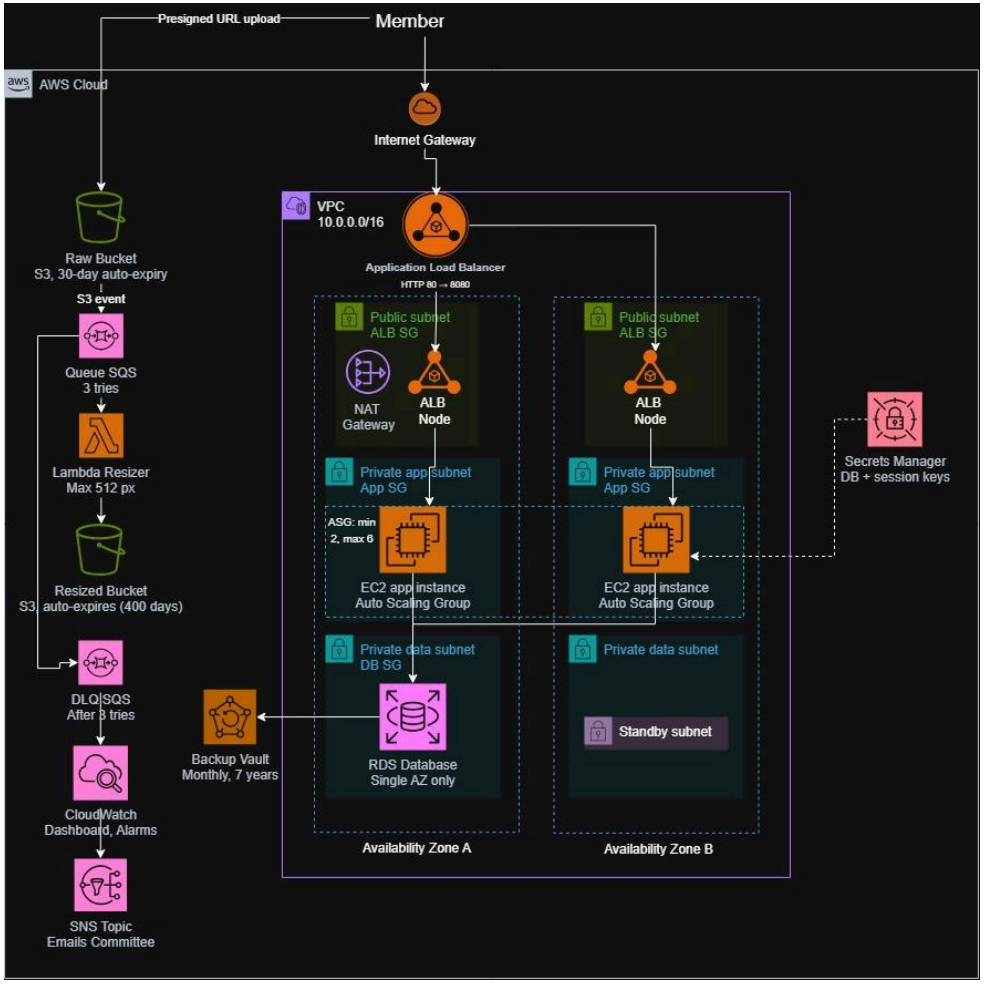
No	Portal Requirements	Source	Assumption
	usage, number of photos waiting to be resized and database connections. An alarm fires when the CPU is high and another will fire when the photo fails processing three times. Alarms move from OK to ALARM, and send a plain-text language SNS email to the volunteer committee's shared mailbox within 5 minutes		
17	The whole solution deploys from one CloudFormation/Terraform template that reaches "CREATE_COMPLETE" from an empty account on a single run in one Learner Lab region. It uses only Learner Lab foundation services and the LabRole/LabInstanceProfile, and can survive a documented breakdown and redeployment	DC1, Lab: "Only us-east-1 and us-west-2 are available.", and "You also cannot create IAM roles, users or groups: where a resource needs permissions, attach the LabRole and LabInstanceProfile that already exist in the account.", Sub: "Marks are available for a template that brings the whole stack up in a single run from nothing, so document the teardown and redeploy you used to prove it."	A9

Assumption Table:

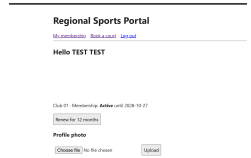
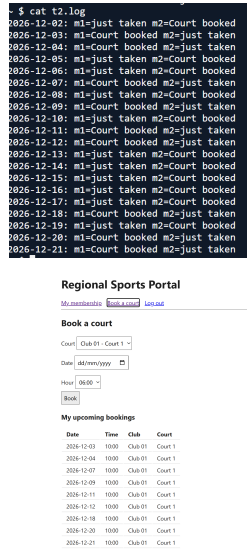
No	Assumption	Source
1	A court is bookable in a fixed 1 hour slot, per court, so a “slot” is the court + date + start time. Bookings also cannot overlap.	This is from “court bookings”, the slot length is a team decision
2	The database enforces one booking per court + date + start time via a unique constraint. The portal will show that the slot is unavailable after confirmation	Team decision
3	Photo storage size is around 5-10MB raw from phones, and resized to a maximum of 512 pixels on the longest side and each member has one profile photo	Gives us as the team real numbers to work with for storage and calculations
4	Bookings will be made synchronous, because the members need an immediate result, and it aligns with the database being able to enforce that only one of the simultaneous requests succeed.	This is a team decision
5	Bookings are held in an RDS because it’s able to support transactions and constraints	Relational database is needed for transactions
6	During peak load, all 4000 members will be active in the registration window, so the target will be around 4000 concurrent sessions, who will each make requests every 10 seconds, which is around 400 requests a second. The load will be generated and tested with k6, and if the generator runs on EC2 it will count towards the 9 instance cap in Learner Lab	Derived from the scenario “about 4,000 members…Almost the whole year's traffic arrives on the two or three Monday evenings when registration opens.”. The request rate and tool (k6) are a team decision
7	Outside the registration windows, members can tolerate up to 60 minutes of portal downtime and 24 hours of recent data loss if an availability zone fails	This is a team decision as outside the three Monday registration evenings, traffic is slow so a short outage will only affect a few members.

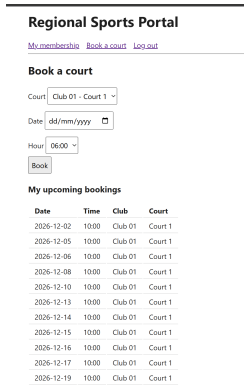
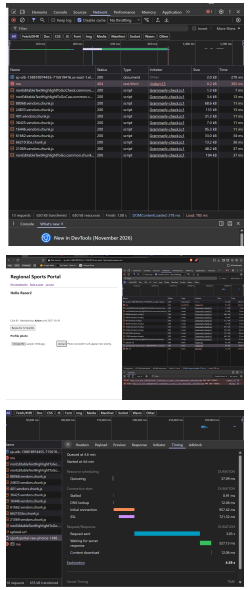
No	Assumption	Source
8	Alarm notifications get sent to an SNS topic subscribed to by the volunteer committee via a shared mailbox, which is written in non-technical English so whoever reads it from the volunteering committee is able to act upon it	Derived from the scenario “no IT staff”, and using a shared mailbox is a team decision because it is simple and avoids relying on a single person
9	For costing, the solution will only run 2 instances as a small baseline capacity for most of the year with 3 registration evenings per year scaling it out to 6 instances for around 4 hours each. The lab environment is only running while testing and is stopped at the end of each day, and the scheduled Monday pre-warm raises the minimum to 4 instances from 4:30-10pm every Monday so that the extra capacity is included in the cost estimate	Derived from the scenario “Almost the whole year's traffic arrives on the two or three Monday evenings when registration opens.”, the instance counts and durations are a team decision so that the pricing calculator estimate can be compared with the actual lab spend
10	Each member registers, renews, books and uploads their own photo through a single shared portal, from their own device. The 12 clubs are not separate systems, each member record has a club attribute. The volunteer committee are not the main source of the load on the portal	Scn: “members doing all of that themselves through a portal.” The single shared portal is a team decision
11	A member chooses their club from a list at registration, and no committee approval is required for this. Member records are kept for 7 years which is enforced by the monthly AWS Backup recovery points so a record is removed any earlier	Derived from Scn: “members doing all of that themselves through a portal.” Choosing the club without approval and keeping backups for 7 years is a team decision
12	Raw uploaded photos are deleted automatically after 30 days once the resized version exists. The resized photo is kept while the membership is active and is deleted after 400 days after upload. Photos are not stored in Glacier and are not subject to the 7 year rule.	Derived from Scn: “member records have to be kept for seven years, though nobody has suggested the photos need keeping that long. ” The 30 day and 400 day durations are a team decision, as 30 days is more than enough to retry a failed resize and short enough to limit storage costs

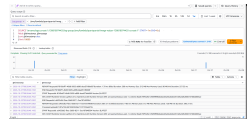
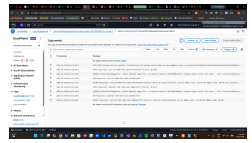
Architecture Diagram: *Figure 1. Deployed architecture.*

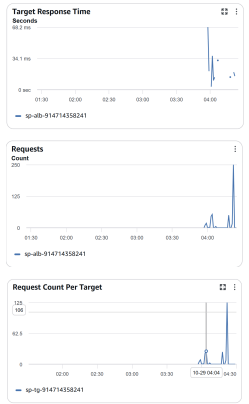


Testing Table:

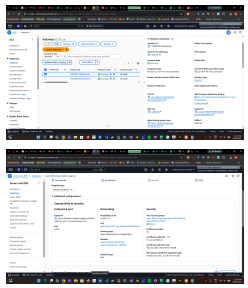
Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
1	R1 Full member journey	Open website link from CloudFormation, → Register new account → Renew membership for 12 months → Book a court slot → Upload profile photo.	A test member completes registration, renewal, a court booking and a photo upload through the portal with no manual intervention from the committee	The test user completed registration, renewed membership, booked a court slot, and uploaded a profile photo successfully. The database and membership status updated automatically.	The primary user workflow functions completely autonomously as designed, replacing manual spreadsheet and email processing.	
2	R2 Simultaneous Booking	Registered two test members, which were race1@example.com and race2@example.com, via the CLI using curl. Executed a shell script which did 20 parallel booking requests for the same court (Court 1) at (10:00 am), which showed output in t2.log	Two booking requests for the same court, date and start time are sent at the same time. Exactly one is confirmed while the other gets a “slot unavailable” message	All 20 Parallel request pairs in t2.log yielded exactly one Court book and one just taken. Browser upcoming bookings for both users confirmed no duplicates were stored.	The PostgreSQL database's unique constraint on the court_id, date and time effectively prevents double bookings under high concurrent traffic.	

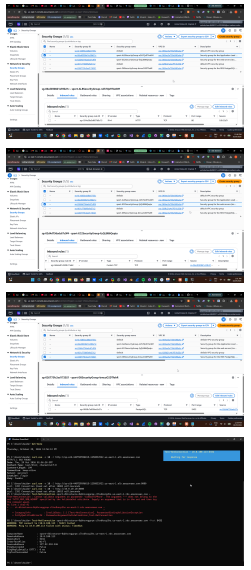
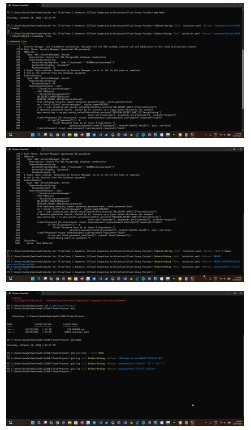
Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
						
3	R3 Upload acknowledged quickly	Logged in as Racer 2, opened F12 with disabled cache checked, then uploaded a 9.5MB file (sample-10mb.jpg). Inspected the upload presigned URL request and direct S3 upload request.	Portal confirms receiving a large photo (5-10mb) in 2 seconds of the file reaching the server/S3, and no resizing occurs in the request path	Presigned the URL returned in 694ms. The 10MB payload went directly to the S3 bucket, returning HTTP 204. The server response time was on S3 under 200ms. Total Duration of 4.54s reflects local bandwidth, not server latency.	Uploading directly to S3 bypasses the web tier entirely, preventing large file transfers from saturating the EC2 bandwidth or blocking server threads.	
4	R4 Background	Upload a raw 5-10mb image via the portal, then	A resized photo that is 512 pixels on	Lambda function sportsportal-image-resize	Asynchronous processing	CloudWatch screenshot of image

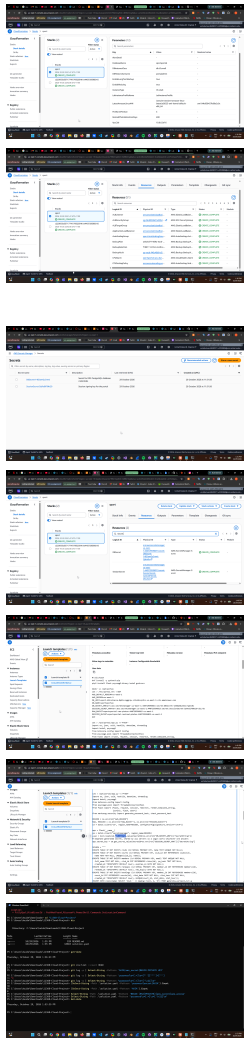
Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
	resize	inspect the /aws/lambda/sportsportal-image-resizer cloudwatch log group in log analytics.	its longest side, appears within 60 seconds of upload, and Lambda log shows it ran	r picked upload07e734cbb1b043b39e21747105e6272c from SQS/S3 and completed resizing in 887.21ms	offloads image manipulation from the web tier without blocking the user, well under the 60-second threshold 887.21 ms vs 60s limit.	resizer 
5	R5 Failed processing	Uploaded a corrupt file being a failure to the raw S3. Monitoring the Lambda execution logs for retry attempts, inspecting the DLQ messages count using AWS SQS get-queue-attributes and verified raw file persistence in S3	A failing job is retried 3 times, then it lands on a dead letter queue. The original upload is still in S3 and DLQ alarm fires	Image resizer lambda threw a Pillow decoding exception on the fake_corrupted_photo.jpg . SQS retried the payload 3 times across visibility timeouts before rerouting to the DLQ . The original file was preserved in s3.	Asynchronous DLQ routing isolates corrupt uploads, preventing poisoned messages from looping endlessly while keeping member upload data enact for review	
6	R6 Member record retention	Queried the CloudFormation backup plan via aws backup get-backup-plan. Triggered an on demand RDS database backup with a 2557 day lifecycle retention via aws backup start-backup-job. Verified vault recovery point using	The AWS backup plan has a monthly rule that keeps recovery points for 2,557 days and the vault holds a recovery point in the database	Get-backup-plan output confirmed DeleteAfterDays: 2557. On demand backup execution returned a valid BackupJobId and successfully added a recovery point to the vault	Aws backup enforces the 7 year constitutional retention requirement automatically, strong recovery points in a managed vault	

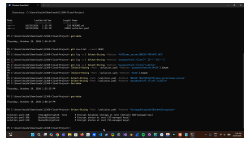
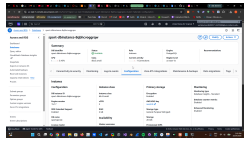
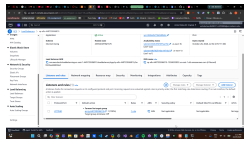
Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
		aws backup list_recovery-points-by-backup-vault			independent of the primary infrastructure stack.	
7	R7 Photo retention	**Steps Taken:** Executed aws s3api get-bucket-lifecycle-configuration on both the raw uploads bucket (sportsportal-raw-photos-138818974455-us-east-1) and resized photos bucket (sportsportal-resized-photos-138818974455-us-east-1).	The raw photo bucket has a 7 day expiry rule in place, while the resized photos have a 400 day after lapse expiry rule. No photos will be transferred to Glacier	Raw bucket JSON confirmed the Expiration for 7 days, resized bucket json confirmed for expiration days 400. Neither bucket configuration contained a transition block	Automated S3 lifecycle rules manage long term storage costs by purging temporary raw uploads after 7 days, avoiding unnecessary glacier transition fees for short lived assets	
8	R8 Load Balancing across AZ's	Performing a load test with the function in CloudShell seq 1 2000 | xargs -P 20 -I{} curl -s -o /dev/null http://$ALB_DNS/ Providing 250 test "users" instantly	During a load test, the ALB's healthy targets will include instances in both AZ's and both will show request traffic	The user spike hit the system and the load balancing was able to respond within 0.018ms per user request allowing all of the users into the system	The Application load balancer is healthy and able to provide users with their request with a split of exactly 125 users per target group.	
9	R9 Peak load	Registered	At 4000 concurrent	K6 summary confirmed	Dynamic scaling	

Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
	and scaling	loadtest@example.com . installed k6 in the cloud shell and executed a 6 minute ramping load test. Monitored Auto Scaling group instance capacity, average CPU and target tracking metrics in cloudwatch	sessions, the error rate will stay under 1% 95 percent of requests will complete under 2 seconds. The ASG will scale out when average CPU exceeds 60% and return to baseline within 30 minutes after load ends	error rate <1% and p95 latencies under 2000 ms threshold. THe Auto scaling group detected CPU exceeding 60% and scaled out from 2 to 4+ instances calling back down after load ceased	policies successfully handle bursty traffic spikes during peak registration windows while maintaining response .	
10	R10 Lab limits	During T9 peak load, executed `aws ec2 describe-instances`, `aws rds describe-db-instances`, and `aws autoscaling describe-auto-scaling-groups` in CloudShell to audit resource quotas.	During peak test, running EC2 instances never exceeds 9, total vCPU never exceeds 32, no instance is ever bigger than “large” and RDS is no larger than “medium”	Command outputs confirmed active EC2 web instances capped at 6 `t3.small` instances (12 vCPUs total). The RDS database ran as `db.t3.small`. ASG capacity constraints verified as Min: 2, Max: 6.	The stack operates strictly within AWS Learner Lab budget and hard quota constraints throughout operational scale-out events.	
11	R11 Instance failure	Exported target instance ID `i-0aafad785fdad429c` and executed `aws ec2 terminate-instances` in CloudShell.	One app instance is terminated during traffic, the portal will keep responding via the other instances and the ASG will bring a	`terminate-instances` confirmed `i-0aafad785fdad429c` entered `shutting-down` state. The subsequent `curl` command returned	Multi-AZ EC2 deployment behind an Application Load Balancer delivers continuous fault tolerance,	

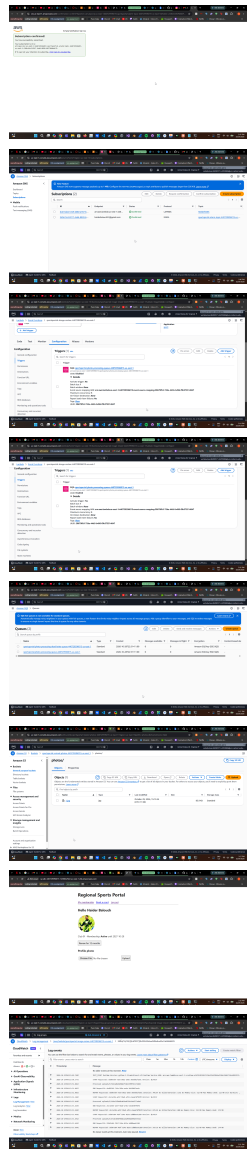
Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
		Immediately issued an HTTP HEAD request using `curl -I $ALB/login` to test load balancer failover and portal availability.	replacement into service within 5 minutes	`HTTP/1.1 200 OK` instantly, confirming the ALB redirected traffic seamlessly to the surviving web node.	ensuring member registration stays online even if a whole Availability Zone or host fails.	
12	R12 Single AZ database recovery	Queried active RDS database snapshots using `aws rds describe-db-snapshots` and inspected AWS Backup vault recovery points using `aws backup list-recovery-points-by-backup-vault`.	Database snapshots exist within the 24-hour RPO window, and recovery points are stored with 2,557-day retention.	Command output confirmed an active RDS snapshot (`awsbackup:job-48057384...`) in `available` status. The vault recovery point confirmed `Status: COMPLETED` with `DeleteAfterDays: 2557` (expiry `2033-10-29`).	Single-AZ RDS deployment in the Learner Lab is backed by automated AWS Backup snapshots, guaranteeing compliance with 24-hour RPO limits and 7-year constitutional retention.	
13	R13 Nothing is reachable that shouldn't be	In the EC2 console, listed running instances and checked their public and private IP address and subnets → In RDS console, checked databases "Publicly Accessible" setting and security group → In EC2 security groups, checked inbound rules of ALB,	EC2 and RDS have no public facing IP's, so direct connection to them from the internet either fails or times out. Only the ALB responds to internet traffic, while the security group rules show the ALB, then	Both web servers have no public IPv4 addresses or public DNS and are both in private subnets. RDS shows "No" to being "Publicly Accessible". The ALB group allows TCP 80 from 0.0.0.0/0, the web server group allows TCP 8080 from only the ALB group, and the database	Only load balancer accepts traffic from the internet. The web servers and database have no public IP address and each security group only accepts traffic	

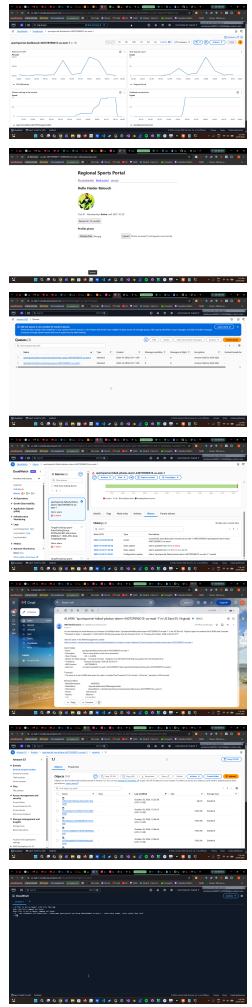
Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
		web server and database groups to check which sources allow which → A laptop on public internet ran curl.exe on Windows Powershell against ALB on port 80 and 8080 and web server private IP, and then Ran Test-NetConnection on database endpoint on Port 5432	the app tier, then the database	group allows TCP 5432 only from the web server group. From the internet, ALB on port 80 returned HTTP 302 to /login, while the ALB on port 8080, web server private IP and database on 5432 all timed out, so no test connected to where it wasn't supposed to	from the layer in front of it, meeting the requirements of DC3 and R13.	
14	R14 Credential handling	Searched template file for words password, secret and AKIA, and then searched for private keys and quoted password values → Cloned project Git repository and searched all 17 commits for access keys, private keys and quoted password values → Checked CloudFormation's deployed stack parameter tab for any	Searching the template, code and repo will show no password or key, as the database credentials exist in the Secrets Manager, and the app connects to it at runtime	The template contains 36 lines containing the word password or secret. Each one is a resource name, {{resolve:secretsmanager …}} reference, a comment, form field for login or code that reads the secret at runtime. There were 0 matches for AWS access keys, none for private keys or for quoted password values. All 17 commits in the Git history also returned no	The database password and session signing in key are both generated by Secrets Manager and passed to the database and servers by reference, so no credentials appear in the template, repository history or the server	

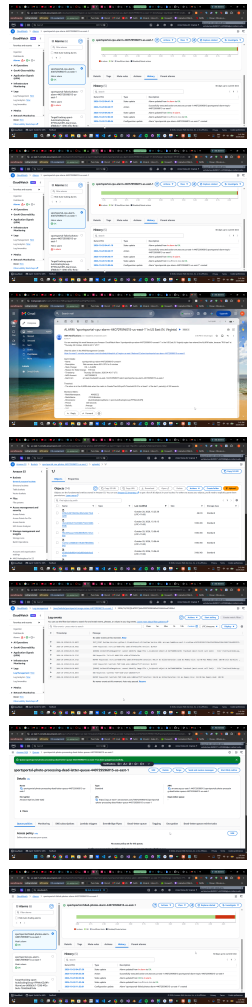
Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
		password parameter and its resources tab for the Secret Managers secrets → Checked Secrets Manager database secret and session secret exist and read their last retrieved dates → Checked EC2 launch template to see what configuration the web servers are given and how application reads credentials		matches for access keys, private keys, or quoted value passwords. Deployed stacks has 12 parameters in which none have a password, and contains two Secrets Manager secrets. Both show the last retrieved date of 29th October 2026 (UTC). The web server launch template user data contains only secret ARN's and application code calls get_secret_value at start to read both secrets	configuration. The last retrieved dates show the running application does read secrets at runtime, so DC6 and R14 are met. One weakness we spotted was that secrets never rotate, and shared LabRole can read both so a compromised server could read the database password. A fix would be to implement automatic rotation on the database secret, and give each component a narrow scoped role in a full account, and run a secret scanner such as gitleaks on every push so the search is repeated rather	

Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
					than done once	
15	R15 Managed storage and encryption	Searched the project “solution.yaml” for encryption settings → In RDS console, opened database Configuration tab and read its encryption setting → In S3 console, opened Properties tab of the raw photo and resized photo buckets and read the Default encryption settings → In EC2 console, listed all EBS volumes to see what disks the web servers have and opened the load balancers Listeners Tab to see which protocols it accepts	All persistent data is in RDS and S3, both show encryption at rest enabled, and no durable data exists on instance disks. The portal is HTTP-only, which is a known weakness	The template sets “StorageEncrypted: true” on the database and “BucketEncryption” on both photo buckets. The deployed database shows “Encryption: Enabled”, using AWS-managed aws/rds key. Both buckets show default encryption with Amazon S3 managed keys. The account only has two EBS volumes, one attached to each of the two web servers with no additional data volumes and both showing “Not encrypted”. The load balancer has a single listener, HTTP:80 forwarding to target group with no HTTP listener	All persistent data is held in managed services such as RDS and S3 and is encrypted at rest so R15 and DC4 are met. The web server's only two disks are two root volumes which hold the operating system and application code installed on boot. No member data, bookings or photos are stored on them, and servers can be terminated and replaced without losing anything (see test 11). The root volumes are not encrypted which does not breach the requirement but it is a gap,	  

Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
					the fix being to set “Encrypted: true” on launch templates volume. Traffic between members and load balancer is HTTP only as lab does not provide domain or certificate which is a known weakness recorded under the Security pillar	

16	R16 Dashboard and alarms	Set the stacks “Alert Email” parameter to a team member as a stand in for committee shared email inbox and confirmed SNS email subscription → Uploaded a “fake.jpg” to test portal resize failure, observe dead letter queue, failed photos alarm history and inbox → Forced CPU alarm into ALARM from CloudShell, then checked its history and inbox → Purged dead letter queue and checked the failed photos alarm returned to OK	The dashboard shows request counts, CPU usage, photos waiting to be resized and database connections. A triggered alarm changes from OK to ALARM and an email is sent to the committee’s shared email within 5 minutes	The dashboard shows in all four widgets such as web server CPU, ALB request count, database connections, and photos rising and falling. Fake.jpg was uploaded at 3:23pm and by 3:33pm the dead letter queue held 1 message, the failed photo alarm went from OK to In Alarm, at 3:35pm, SNS was called in the same second, and email arrived in the same minute, about 13 minutes after upload. The CPU alarm was in ALARM mode at 3:40pm, sent its email in the same minute, and returned to OK at 3:41pm when CloudWatch re-read the real CPU. After the dead letter queue was purged at 3:50pm, the failed photos alarm returned to okay at 3:57pm. Both emails use the default CloudWatch format, one readable description followed with technical detail	The dashboard and alarms both work and each alarm sends an email within about a minute of changing state, so R16 is partially met. We found out that committee members will find out about a failed photo around 13 minutes after upload, the 3 retries take about 3 minutes each and the alarm waits for a full 300 second window so the 5 minute target is held from the moment the alarm fires, not from the fault. To fix it we can set it to “Period: 60” on both alarms. The emails aren’t in plain english so A8 isn’t met, and to fix it we can rewrite the emails as	

Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
					instructions the committee can act on. The CPU alarm was tested by force as the real CPU usage didn't cross 0.32% so real load hasn't been tested. Finally, the testing showed that the resizer trigger was disabled in the console causing 13 photos to be unprocessed, and no alarm fired because the failed photos alarm only watches the dead letter queue. To fix it we can add an alarm on the main queue's oldest message age and make it so a redeploy will restore the trigger	

Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
						

Test	Requirement Tested	Steps Taken	Expected result	What actually happened	What it told us	Evidence of Result
17	R17 Single template deployment	Queried stack deployment status in AWS CloudShell using `aws cloudformation describe-stacks --stack-name Sport --region us-east-1`.	The CloudFormation template deploys the entire infrastructure stack from an empty account state to `CREATE_COMPLETE` in a single run.	Command output confirmed stack `Sport` reached `CREATE_COMPLETE` status with zero resource creation failures.	The Infrastructure as Code template is completely self-contained and idempotent, satisfying design constraint DC1 and requirement R17.	

Chosen Services:

Compute and Scaling: EC2 instances in an Auto Scaling group behind an ALB meet the two AZ constraints. We chose an ALB over a Network Load Balancer because it health checks at the HTTP level and reports request counts by AZ in (R8). Scaling uses 3 signals: average CPU at 60%, requests per target at 300 and a scheduled action that raises the minimum for 4 servers from 16:30 to 22:00 every Monday AEST time. New servers install their package at boot and take minutes to become healthy, which is too slow to rely on reactive scaling alone.

Direct upload: If 4,000 members each sent a 7.5mb photo through the web tier, about 30 GB would pass through the servers in the same peak as the bookings. A signed upload form sends the bytes to S3 instead and limits each file to 25MB and 5 minutes.

SQS and Lambda: Resizing in the request would cost about 0.9 AWS Compute per photo (887ms in the Lambda log, T4), nearly half of the 2-second budget in R3 before the upload itself is counted, and would hold one of each server's 8 request threads for that time. We also rejected S3 triggering Lambda directly: SQS gives exactly three attempts, buffer bursts (we cap Lambda at 5 concurrent runs) and exposes dead letter depth as an alarm.

RDS PostgreSQL. DynamoDB conditional writes could also stop double bookings, but members, renewals and bookings are relational; the committee will want reports and a unique constraint on court, date and hour states the rule directly. This cost is the single AZ risk in section

AWS Backup: Automated backups last around 35 days. A monthly AWS backup rule keeps recovery points for 2,557 days in a vault, and the template retains when the stack is deleted, so records outlive the stack.

Secrets Manager: The brief asks for a secret service. Secrets Manager generates the passwords, so nobody sees or commits them and a second secret signs session cookies identically on every server.

NAT gateway: Servers need outbound access at boot to install packages and read secrets, and one NAT gateway is the simplest route. It costs an hourly fee plus data charges and sits in one AZ

Pillow library: The resizer needs the Pillow image library, which Lambda does not include. A helper function in the template downloads it so deployment stays a single run (DC1). If the download fails, a parameter turns resizing off, and photos are stored unresized.

Monthly Estimates:

The pricing calculator estimate is $245.47 per month in US East (N Virginia / us east) 1 region, under the assumption 9: two servers all year, four servers from 16:30 to 22:00 every Monday, and six servers for about 4 hours on each 3 registration evenings. The Monday pre-warm adds about 670 server-hours per year (2 extra servers time 5.5 hours times 52 Mondays), roughly 48 hours a month. With the baseline (2 servers x 730 hours) and about 2 hours a month of surge, that is about 1510 server hours a month, which is the figure to enter in the calculator

Fixed items dominate: The load balancer, NAT gateway and the database and the two baseline servers all charge every hour regardless of traffic. S3 is small: at most about 30 GB of raw photos (4,000 members x 7.5 MB) before the 30-day expiry; the resized copies are tiny. Lambda and SQS, and SNS are negligible at this volume (Confirm in the calc). Secrets Manager charges for two secrets, AWS backup for stored recovery points and CloudWatch for one dashboard and the alarms.

Lab spend was $26.16 AUD over 7 testing sessions. The AWS Pricing Calculator estimate of $245.47 AUD/month assumes always-on running, whereas the lab ran only while testing. To reconcile them, we multiplied the calculator's hourly rate of $0.34 AUD/hour by the 62 hours actually run, giving $21.13 AUD, against an actual spend of $26.16 AUD; the difference of $5.03 AUD comes from load testing data transfer spikes during k6, and parallel curl runs, ALB capacity units, and retained EBS volumes and RDS snapshots while instances were stopped. The us-east-1 region and single-AZ constraints affect cost too: us-east-1 prices are roughly 20–25% cheaper than Sydney (ap-southeast-2), Multi-AZ RDS would roughly double the database instance charge (adding ~$73.84 AUD/month), and a second NAT Gateway adds another hourly fee (~$48.42 AUD/month).

Pillars:

Security:

1. Strengths: No server or database has a public address; security groups chain from the ALB to the web servers to the database; the database and a photo bucket are encrypted at rest. Passwords are generated by Secrets Manager and never appear in the template.
2. Weakness: The portal is HTTP only because the lab provides no domain or certificate, so logins and personal details cross the internet unencrypted.
3. Fix: Register a domain, issue a certificate in AWS Certificate Manager, add an HTTPS listener to the ALB, redirect port 80 to it and mark the session cookie secure. The LabRole, which every resource must use, also grants far more access than needed; production would use a narrowly scoped role for each component.

Reliability:

1. Strengths: The web tier spans two AZ’s and the group replaces a terminated server (T11), failed photo jobs are retired and preserved in the dead letter queue, and monthly backups outlive the stack.

2. Weakness: The database is a single instance in 1 AZ, and 1 NAT gateway in the first AZ serves both private subnets, so an outage will leave the new server unable to install packages or read secrets (state the effect found in testing).
3. Fix: From a full account, enable Multi-AZ RDS, add a NAT gateway and route table per AZ and use VPC endpoints for S3 and Secrets Manager. Meanwhile, we tested a restore and recorded its duration (Section3).

Performance Efficiency:

1. Strengths: Uploads go straight to S3, and resizing runs in the background so the web tier handles only small requests; 3 scaling signals and a Monday pre-warm cover the predictable peak.
2. Weakness: Scaling is still reactive, and new servers install their software at boot, so extra capacity arrives minutes after the load does (T9{observation}). The request count target of 300 also looks low, as we read the metric it is counted per target per minute. So the group may jump straight to 6 servers (confirm from the T9 scaling graph.
3. Fix: Bake the software into a machine image so servers start healthy in seconds and tune the target from measured data.

Individual reflection, 400 words each

Haider:

I wrote the requirements table, the assumptions table, the source key, and I drew the architecture diagram in draw.io. Each requirement was linked back to either the scenario, design constraint, and/or lab restriction in some way, shape or form. The assumptions were implemented to answer for decisions in the brief that were left open such as booking time slots, concurrent sessions, and photo retention periods. The diagram shows both availability zones, public, application and database subnets, and direction of data flow which includes an asynchronous photo path.

The technical problem was with my documentation and diagram, as it was describing a different system from the one that was deployed with the template. I had written it with the design discussions in mind, while Ben was building the template separately. Once the template was fully completed, I compared the requirements and diagram and tried to find in the template where those values would be and what they were. Some differences included the template showing 6 maximum instances and the diagram showing 7, template expiring raw photos after 7 days when the document said 30, and two assumptions describing an admin role and club changing page for applications that aren't present in the template, just to name a few. For most differences it was easier to edit the report than the template, however I asked for the template to change the raw photo expiry to 30 days as it was our group decision.

What I would do differently is to not write numbers into the report, diagram and tests by hand but instead work alongside the template. I would see which values are fixed and changed, and edit the report alongside it accordingly. I did this comparison at the end, and was only aware of the changes very recently, which took the better part of a day to completely rewrite the report and make sure each requirement, assumption, and test was matched correctly to one another. Going into another project next time, I will agree with whoever is building the template on which values are fixed and being changed, and keep them as a small list inside the group report, so that I am able to check it against the report afterwards so that mismatches will take a couple minutes to change rather than a whole day.

Zubair:

In this project, my role was building, running, and documenting our test suite for Tests 1 through 10 (T1–T12). I ran tests covering the complete member portal workflow, wrote shell scripts for parallel curl booking requests to test database race conditions (T2), measured direct-to-S3 upload speeds (T3), tracked asynchronous Lambda resizer logs (T4), and ran k6 load testing while auditing Learner Lab resource caps (T9–T10). Beyond testing, I wrote the Chosen Services section, reconciled our AWS Pricing Calculator estimate ($245.47 AUD/month) against our actual lab spend ($26.16 AUD over 62 hours), completed the Well-Architected Review for

Security, Reliability, and Performance Efficiency, and co-created and presented our team's slide deck.

A major technical challenge I faced was during Test 5 while trying to trigger the asynchronous failure path and Dead-Letter Queue (DLQ). When uploading test text files or uploading directly using aws s3 cp, the DLQ count remained at 0, and no error logs appeared. To diagnose this, I checked our bucket event configuration using aws s3api get-bucket-notification-configuration and queried queue metrics with aws sqs get-queue-attributes. This revealed two separate issues: direct CLI uploads skipped the portal's presigned URL workflow, and CloudShell Bash history expansion (!contains) was breaking my queue URL variables into blank strings. Additionally, browser validation rejected non-image files. I solved this by using printf to generate a corrupted JPEG containing a valid header, uploading it through the portal UI, and putting scripts into nano to avoid parameter expansion errors. I then waited out the 180-second SQS retry timeouts until the DLQ count successfully updated to 1.

Reflecting on my end, I would change how I time and sequence testing. I initially treated verification as a single task at the very end of development. This created unnecessary pressure when debugging CLI syntax, variable expansion errors, and retry delays before the project deadline. Next time, I will adopt a continuous testing approach, writing and running small test scripts as each CloudFormation resource is added. Testing resources step-by-step during stack construction will catch deployment issues early and keep documentation clean and accurate throughout the project.

Ben u3261492:

The part of the solution that I personally worked on was the yaml file and github repository for the project. after our requirements table was made up during group discussion and our first meeting where we discussed everything that we would need for our solution to be able to fulfill the needs of the project I sat down and developed both the AWS service requirements in the YAML as well as the basic website that shows the production of what was made. The biggest issue that I faced technically was the raw bucket that accepted the asynchronous upload of photos needed permission to send the photo to the resizing queue but the queue policy referred to the bucket, the code was originally hardcoded with `aws:SourceArn` and `DependsOn: PhotoProcessingQueuePolicy` this meant that there was a circular dependency. The solution to this problem was to look at the CloudFormations "Events" tab to show resource failure and then see that I needed to construct the name and not reference it in order to avoid that dependency.
Additionally when the team got together and we decided on all of the constraints with all of the original time frames for keeping information each of us was separately writing down the parts that we thought were important and a mix up in a number of time frames occurred. For example when discussing the raw photo storage time we bounced the idea of keeping those files for 1 day, 7 days and 30 days. Despite each of us agreeing on 30 days I had only written down 7 and as such wrote the code to be 7 days. This caused some minor issues when reviewing the code and the requirements later on. It is worth having one master document moving forward such that

when deciding on important requirements that tuning out as an individual does not cause a major issue with a code's foundations.
Example:

Requirement	Value
Raw Photos Held	30 Days
Maximum Users expected	4 000
Etc	etc